CPP LOB Prerequisites

Ananya Prabhav

October 2026

§1 Introduction

- We can think of a financial market as an electronic auction operating at milliseconds. A **bid** is the highest price a buyer is willing to pay for an asset. An **ask** is the least price a seller is willing to accept for the asset. **Spread** is the difference Spread = Ask − Bid.
- A **Limit-Order Book** is a real time data-structure managed by an exchange that maintains all active, unexecuted buy and sell orders sorted by price and time. Most electronic exchanges process incoming market orders using *Price-Time Priority*. The best price gets filled first; if two orders have the same price, the older order gets executed first.
- **Market Making:** Market makers (like Optiver etc.) continuously place both Bid and Ask orders simultaneously. They provide liquidity (see the next point).
  Explaining with Gold Store Owner analogy: Imagine you open a shop that buys and sells gold coins. If someone walks in and says, "I need cash urgently, I want to sell my gold coin right now," you don't tell them: "Let's put a sign on the window and wait 3 weeks for another customer to walk in who wants to buy it." Instead, as the shop owner, you provide an immediate service. You say: 1. "I am *always* ready to buy a gold coin from you right now for $95." (This is your Bid) 2. "I am always ready to sell a gold coin to anyone right now for $105." (This is your Ask) By standing in the middle, continuously willing to both buy and sell, you are a Market Maker. You aren't betting on whether gold prices will go up or down tomorrow; you are simply providing a continuous two-sided marketplace. Market makers do this, but instead of with gold coins, they do it electronically with thousands of stocks, options and future contracts every millisecond using automated (C++) algorithms.
- **Liquidity** is simply the ease with which you can convert an asset into cash (or vice versa) instantly without causing a massive price swing. 1. High Liquidity example: Apple stock. If you own 1,000 shares of Apple, you can click "Sell" on your brokerage app and convert them to cash in 0.05 seconds at almost the exact current market price. 2. Low Liquidity example: A rare vintage mansion or a tiny, obscure cryptocurrency. If you want to sell it instantly, you might have to slash the price by 30% because there are no buyers around. Without liquidity, markets are clunky and inefficient. An investor can't wait weeks to buy shares of a company. Market makers keep the market active and functioning. Market makers make money primarily through the bid-ask spread. Like if we bought for $95 and sold for $105 we made the $10 spread profit. If you do this thousands/millions of times a day, the small profits compound.
- However there is a catch/risk. What if we buy a gold coin for $95 and suddenly a gold reserve is found and the price of gold crashes to around $80. Then the market makers are forced to sell for a loss. This is called **Inventory Risk**-the risk that the market price moves against the inventory you are holding before you can flip it. This is why HFTs need low latency programmers, so that our algorithms adjust very quickly to changes in the market. **Inventory:** The total number of shares/contracts a market maker holds. We can classify our inventory into positive and negative. Positive is the usual: you have something and you wanna sell it. Negative is when we borrowed a share and sold it, and we promised from who we borrowed that we'll buy it. Its kind of like a normal loan (but there is no interest-rather we wanna buy at a lower price).
- **PnL** (Profit and Loss): 1. Realized PnL-cash made from closed trades. 2. Mark-to-market (unrealized) PnL-value of currently open inventory based on the current market mid-price.
  So Total PnL = Cash Balance + (Inventory × Mid Price).

§2 Plan

To build a Limit Order Book essentially means that we want to build a high speed engine which does the following:

- Receives orders: `Add(OrderID, Side[Buy/Sell], Price, Quantity)` or `Cancel(OrderID)`.
- Stores active orders in two sorted lists: `Asks` (in ascending order) and `Bids` (in descending order).
- Executes a match whenever an incoming BUY price $\geq$ best Ask price, or an incoming SELL price $\leq$ best Bid price.
- Outputs completed trade records `(Price, Quantity, BuyerID, SellerID)`.

This project is to simplistically build the logic of a matching engine. I use a `std::vector<>` instead of `std::map<>`, and don't use priority queues but just iterate. However the price-time priority rule is never compromised. We can just keep it (the OB) unsorted, and match the orders with time; this approach indeed works we can argue.

§3 Different Order Types (that I implement here)

- **Market order:** An order to buy or sell immediately at the best available current price in the order book. Execution guarantee, but no price guarantee.
- **Limit order:** An order to buy at a specified maximum price (or lower) or sell at a specified minimum price (or higher). Price guarantee, but no execution guarantee.
- **Stop order:** A stop order is of two types - stop-limit or stop-market. We need to specify which one among those. Now a stop order remains dormant until the market's mid-price for that stock goes below a "trigger price" or above a "trigger price". We can choose the settings we wanna set to. After the trigger goes off, the stop-x order converts to x, $x \in \{\text{Limit},\ \text{Market}\}$.
- **Good-till-cancelled (GTC) order:** Indicates that the order remains active in the exchange's order book until it is either fully executed or explicitly cancelled by the trader (though brokers usually enforce a hard ceiling, such as 30 to 90 days).
- **Fill-or-Kill (FOK) order:** An execution instruction combining an All-or-None quantity constraint with an Immediate-or-Cancel time constraint.

Also there's another thing I haven't accounted for in the code. We can have GTC together with limit order - we can combine certain types. However I've assumed one special characteristic comes alone, not composed with others.

More details of the code I've mentioned as comments in it. Note that in the code I used the last trade price for triggering Stop orders, not mid price. The stop order mechanics was hard. We use cascading triggers. A stop order sits quietly until the "last trade price" crosses a certain threshold. When it activates, it enters the market as a regular order and executes. If that execution happens to push the market price far enough, it can trigger another resting stop order, which then executes and pushes the price further, triggering a third one.